OpenClaw: Documento Técnico

Arquitectura, Seguridad y Despliegue en Producción

Versión: 1.0
Fecha: Marzo 2026
Audiencia: CTOs, Arquitectos de Software, DevOps Engineers, Security Engineers

---

1. Objetivo y Alcance

Este documento describe la arquitectura técnica, componentes, patrones de despliegue y buenas prácticas para ejecutar el ecosistema **OpenClaw** (Core + ClawHub + Control UI + Webclaw + SwarmClaw + alternativas de seguridad) en entornos empresariales, con foco en VPS Linux y contenedores Docker[1][2]. Cubre desde la topología recomendada hasta consideraciones de seguridad, observabilidad y operación continua en producción[3][4].

---

2. Arquitectura Lógica de OpenClaw

2.1 Componentes Principales

OpenClaw Core + Gateway

Proceso principal Node.js que conecta canales (WhatsApp, Discord, Telegram, Slack, etc.), ejecuta skills y orquesta llamadas a LLMs[1][5]. Incluye un Gateway HTTP/WebSocket que expone API REST y la Control UI, por defecto en el puerto 18789[2][6].

Control UI

Dashboard web nativo para:

- Configurar proveedores LLM

- Gestionar canales de mensajería
- Ver sesiones, uso de tokens, cron jobs y logs en tiempo real[2][7]

ClawHub (Skills Marketplace)

Marketplace con 3.000+ skills comunitarios y aproximadamente 1,5M de descargas acumuladas en 2026[8][9]. Cada skill define un `SKILL.md` (descripción, inputs/outputs, permisos) y el código de implementación[10][11].

Webclaw

Dashboard web externo que genera UI automática para skills leyendo SKILL.md y otras señales (tablas, introspección de respuestas, overrides en `ui.yaml`)[12][13].

SwarmClaw

Orquestador multi-agente que se conecta a instancias de OpenClaw y a 14–15 proveedores LLM para definir agentes, workflows y scheduling desde una UI centralizada[2][14][15].

NanoClaw e IronClaw (Seguridad)

- **NanoClaw**: Reimplementación minimalista (~500–700 líneas) con aislamiento fuerte por contenedor Docker por chat, para maximizar auditabilidad[16][17]
- **IronClaw**: Reescritura en Rust con sandboxing WASM por tool, vault de credenciales cifrado y ejecución en TEEs para entornos regulados[16][17]

2.2 Topología de Referencia

En un entorno de producción típico:

Capa de entrada:

- Reverse proxy (Traefik o Nginx) con TLS y HTTP→HTTPS redirect[6][18]
- Opcional: DNS + Tailscale Serve para exponer solo loopback del Gateway[19]

Capa de aplicación:

- Contenedor(es) de OpenClaw (Core + Gateway)
- Webclaw como servicio adicional si se requiere UI auto-generada
- SwarmClaw como orquestador opcional[3][15]

Capa de datos y observabilidad:

- PostgreSQL para persistencia (según configuración)
- Redis para caché/colas (si se usan patrones de colas)
- Vector store (Qdrant, Weaviate, etc.) para memoria semántica[6]
- Prometheus + Grafana + stack de logs (Loki/ELK) para métricas y logging centralizado[4][20]

---

3. Modelos de Despliegue

3.1 Instalación Rápida con Script Oficial

Los docs oficiales recomiendan comenzar con el installer script para validar configuración y canales[2][21].

Linux/macOS/WSL2:

curl -fsSL https://openclaw.ai/install.sh | bash -s -- --no-onboard
openclaw onboard

El comando `onboard` guía:

- Configuración de LLM principal (API keys)
- Alta de canales (ej. Telegram, Discord, etc.)
- Creación de la carpeta de estado y configuración[2][22]

Este modelo sirve para desarrollo/local, no es el ideal para producción continua.

3.2 Despliegue en VPS con Docker

La mayoría de guías actualizadas recomiendan empaquetar OpenClaw en Docker y correrlo en un VPS Linux (Hetzner, Hostinger, DO, etc.), con o sin Traefik[23][24][25].

Patrón típico:

- Contenedor `openclaw/openclaw:latest`
- Volumen persistente para datos (`/app/data` o similar)
- O bien exponer directamente `18789` o usar un reverse proxy en otro contenedor que hable con el Gateway[24][18]

Ejemplo minimal:

```
services:
openclaw:
image: openclaw/openclaw:latest
restart: unless-stopped
env_file:
- .env
volumes:
- ./data:/app/data
ports:
- "18789:18789"
```

3.3 Despliegue con Reverse Proxy y TLS

Guías específicas para Hostinger/Cherry Servers muestran la combinación Docker + Traefik/Nginx para servir OpenClaw en HTTPS[18][24][26].

Traefik:

- Entrypoints en 80/443
- Certificados Let's Encrypt con HTTP-01
- Regla de host apuntando al servicio OpenClaw (puerto interno 18789)[18][23]

Nginx:

- VirtualHost que hace `proxy_pass http://127.0.0.1:18789`
- Certificados con certbot[24][27]

Beneficios:

- Cierre de puerto 18789 al exterior (solo accesible desde el proxy)[28][29]
- Terminación TLS centralizada (más simple renovar/rotar certs)
- Posibilidad de añadir WAF/rate limiting a nivel del proxy

3.4 Despliegue con Dokploy u Otras PaaS Self-Hosted

Plataformas como Dokploy permiten definir apps basadas en Docker Compose y gestionar dominios, certificados y despliegues desde una UI[30][31][32].

Se define un servicio `openclaw` en Docker Compose. Dokploy se encarga de:

- Aprovisionar contenedores
- Asociar dominios a puertos internos
- Emitir certificados TLS automatizados

Esto es útil cuando quieres escalar a múltiples clientes/proyectos sin gestionar a mano cada compose.

---

4. Seguridad

4.1 Riesgos Específicos de OpenClaw

Superficie de ataque del Core:

El proyecto ha crecido muy rápido (cientos de miles de estrellas), con un código base grande[1][33]. Múltiples canales, acceso al sistema de archivos y ejecución de comandos lo convierten en un objetivo atractivo[34][28].

Supply Chain de ClawHub:

Investigaciones de seguridad encontraron más de 300 skills maliciosos en ClawHub, incluido el skill más descargado en un momento dado[35][36]. Estos skills apuntaban a robar claves SSH, wallets y abrir reverse shells[35][36].

Prompt Injection y Exfiltración de Credenciales:

Los agentes que tienen herramientas de shell, HTTP o DB son susceptibles a ataques de prompt injection si no hay controles adecuados[16][4].

4.2 Controles Recomendados

Capa de red:

- No exponer directamente 18789 a internet; siempre detrás de reverse proxy o VPN[24][29]
- Uso de firewalls (UFW, Security Groups) para reducir superficie de puertos visibles[20]

Gestión de secrets:

- Usar variables de entorno o un secret manager en vez de poner claves en el repositorio[29][4]
- Rotar periódicamente API keys de LLMs y tokens de canales

Skills (ClawHub):

- Mantener whitelist de skills permitidos
- Preferir skills populares, auditados y mantenidos (consultar rankings actualizados)[9][37]
- Activar integraciones de escaneo (VirusTotal, análisis estático) cuando el entorno lo soporte[35][16]

Hardening del runtime:

- Ejecutar contenedores con usuarios no-root y `no-new-privileges` cuando sea posible[29]
- Limitar accesos al sistema de archivos del host (montar solo lo estrictamente necesario)[28]

4.3 Uso de NanoClaw e IronClaw

NanoClaw:

Ideal para casos donde el código del agente debe auditarse completo (500–700 LOC) y se requiere aislamiento fuerte por cliente/chat[16][17]. Se usa frecuentemente en entornos con fuerte compliance pero sin requisitos de TEE.

IronClaw:

Orientado a industrias reguladas (finanzas, salud, gobierno) donde se exige:

- Ejecución en TEEs
- Vault de credenciales cifrado
- Sandboxing WASM por herramienta
- Detección de exfiltración de secretos en tiempo real[16][17]

Patrón típico: OpenClaw Core para la mayoría de interacciones y NanoClaw/IronClaw para subdominios de máxima sensibilidad.

---

5. Observabilidad y Operación

5.1 Métricas y Logs

Buenas prácticas de producción recomiendan:

- Exponer métricas (HTTP metrics, LLM usage, latencias) y scrapearlas con Prometheus[4][20]
- Crear dashboards en Grafana para:
  - Request rate y error rate
  - Latencias p95/p99
  - Uso de tokens por modelo LLM[4]
- Centralizar logs del contenedor OpenClaw (stdout/stderr) en:
  - Loki o ELK stack, con etiquetas por entorno/proyecto[20][28]

5.2 Backups y Restores

Si OpenClaw usa bases de datos externas (PostgreSQL, Qdrant, etc.) se recomienda:

- Backups automáticos (ej. cron + `pg_dump` + sync a S3/Wasabi)[6]
- Pruebas de restore periódicas (no solo asumir que los backups funcionan)[6][20]

5.3 Entornos y Despliegues

- Mantener como mínimo **dev**, **staging** y **prod**[20]
- Cualquier actualización de OpenClaw, skills críticos o infra:
  - Se prueba primero en staging (mismas versiones de Docker y dependencias)
  - Se aplica un tipo de canary en producción (por ejemplo, un solo nodo/instancia primero)[4][28]

- Adoptar prácticas de CI/CD:
  - Pipelines GitHub/GitLab que:
    - Construyen imágenes
    - Ejecutan tests básicos
    - Despliegan a staging y luego a prod bajo aprobación

---

6. Guía de Despliegue Técnico

6.1 Pasos de Despliegue en VPS

1. Provisionar VPS Linux (Ubuntu LTS 22.04+, 2–4 vCPU, 4–8 GB RAM)[24][6]
2. Instalar Docker y Docker Compose[23][27]
3. Validar OpenClaw con el installer script en local (opcional pero recomendable) para configurar LLM y un canal de prueba[2][21]
4. Empaquetar OpenClaw en Docker (imagen oficial `openclaw/openclaw:latest`), montar volumen de datos y configurar `.env` con API keys[24][38]
5. Añadir reverse proxy (Traefik o Nginx) para exponer `https://openclaw.tu-dominio.com` con TLS automático[18][23]
6. Configurar Control UI y conectar:
   - Proveedores LLM
   - Canales (Telegram, Discord, Slack, etc.)[2][21]
7. Instalar skills desde ClawHub con política de seguridad:
   - Whitelist + escaneo de malware
   - Revisión de permisos de cada skill[8][35]
8. Añadir observabilidad (Prometheus+Grafana, logging centralizado) y política de backups[4][20]
9. Documentar procedimientos:
   - Onboarding de nuevos agentes/skills
   - Gestión de incidentes y rotación de claves
   - Rollback de versiones de OpenClaw y de infra

6.2 Configuración de Variables de Entorno

El sistema OpenClaw utiliza variables de entorno para configuración[38][2]:

Variable	Descripción
OPENCLAW_HOME	Directorio base de instalación
OPENCLAW_STATE_DIR	Directorio para datos persistentes
OPENCLAW_CONFIG_PATH	Ruta al archivo de configuración
OPENAI_API_KEY	Clave API para OpenAI
ANTHROPIC_API_KEY	Clave API para Anthropic
GROQ_API_KEY	Clave API para Groq

Table 1: Variables de entorno principales

6.3 Configuración de Firewall

Configuración básica UFW recomendada:

```
sudo ufw allow OpenSSH
sudo ufw allow 80,443/tcp
sudo ufw enable
```

Nunca exponer directamente el puerto 18789 sin autenticación adicional[24][29].

6.4 Estructura de Docker Compose para Producción

Ejemplo completo con Traefik:

```
version: '3.8'

networks:
proxy:
external: false

services:
traefik:
image: traefik:v2.10
command:
- "--api.dashboard=true"
- "--providers.docker=true"
- "--entrypoints.web.address=:80"
- "--entrypoints.websecure.address=:443"
- "--certificatesresolvers.le.httpchallenge=true"
```

```
- "--certificatesresolvers.le.httpchallenge.entrypoint=web"
- "--certificatesresolvers.le.acme.email=${ACME_EMAIL}"
- "--certificatesresolvers.le.acme.storage=/letsencrypt/acme.json"
ports:
- "80:80"
- "443:443"
volumes:
- /var/run/docker.sock:/var/run/docker.sock:ro
- ./traefik:/letsencrypt
networks:
- proxy

openclaw:
image: openclaw/openclaw:latest
container_name: openclaw
restart: unless-stopped
env_file:
- .env
volumes:
- ./data:/app/data
labels:
- "traefik.enable=true"
- "traefik.http.routers.openclaw.rule=Host(${DOMAIN})"
- "traefik.http.routers.openclaw.entrypoints=websecure"
- "traefik.http.routers.openclaw.tls.certresolver=le"
- "traefik.http.services.openclaw.loadbalancer.server.port=18789"
networks:
- proxy
```

---

7. Consideraciones de Escalabilidad

7.1 Escalamiento Horizontal

Para entornos de alta carga:

- Múltiples instancias de OpenClaw detrás de load balancer
- Separación de canales por instancia (ej. una instancia por cliente corporativo)
- Uso de Redis como backend compartido para sesiones[6]

7.2 Escalamiento de LLM Providers

Estrategias de optimización:

- Rate limiting por usuario/canal
- Caching de respuestas frecuentes
- Routing inteligente (queries simples → modelos rápidos/baratos, queries complejas → modelos premium)
- Fallback entre providers (Anthropic → OpenAI → Groq)[5][4]

7.3 Gestión de Costos LLM

Provider	Costo aprox. (1M tokens input)	Uso recomendado
Groq	$0.27	Speed-critical, inference simple
OpenAI GPT-4o mini	$0.15	Balance costo/calidad
Anthropic Claude 3.5	$3.00	Tareas complejas, código
Google Gemini 2.0 Flash	$0.075	OCR, procesamiento visual

Table 2: Comparativa de costos LLM (precios 2026 aproximados)

Estrategias de reducción:

- Usar modelos locales (Ollama) para casos no críticos
- Implementar prompt caching agresivo
- Limitar context window cuando sea posible
- Monitorear uso por skill y optimizar los más costosos[4]

---

8. Checklist de Producción

8.1 Seguridad

- ☐ Puerto 18789 no expuesto directamente a internet
- ☐ TLS configurado con certificados válidos
- ☐ API keys en variables de entorno, no en código
- ☐ Firewall configurado (solo 22, 80, 443 permitidos)

- ☐ Skills auditados y en whitelist
- ☐ Escaneo de malware activado para ClawHub
- ☐ Contenedores corriendo como non-root user
- ☐ Logs centralizados sin información sensible

8.2 Operaciones

- ☐ Backups automáticos configurados y testeados
- ☐ Monitoreo con Prometheus + Grafana
- ☐ Alertas configuradas (uptime, error rate, latencia)
- ☐ Documentación de runbooks para incidentes
- ☐ Entornos dev/staging/prod separados
- ☐ CI/CD pipeline funcional
- ☐ Plan de rollback documentado
- ☐ Rate limiting configurado

8.3 Compliance y Gobernanza

- ☐ Política de retención de logs definida
- ☐ Auditoría de accesos implementada
- ☐ Contratos con LLM providers revisados (términos de datos)
- ☐ GDPR/CCPA compliance verificado si aplica
- ☐ Política de incident response documentada
- ☐ SLAs definidos con clientes

---

9. Troubleshooting Común

9.1 Problemas de Conectividad

Síntoma: Control UI no accesible

Verificaciones:

1. Contenedor corriendo: `docker ps | grep openclaw`
2. Logs: `docker logs openclaw --tail=100`
3. Puerto expuesto: `netstat -tulpn | grep 18789`
4. Firewall: `sudo ufw status`
5. DNS resolviendo correctamente

Síntoma: Canales no conectan (Telegram, Discord, etc.)

Verificaciones:

1. API tokens correctos en `.env`
2. Webhooks configurados correctamente
3. Logs del contenedor para errores específicos
4. Network connectivity del contenedor a internet

9.2 Problemas de Performance

Síntoma: Latencias altas en respuestas

Causas comunes:

1. LLM provider con throttling o slow response
2. Context window muy grande (reducir historial)
3. Skills mal optimizados con loops o timeouts largos
4. Falta de recursos (CPU/RAM) en VPS

Soluciones:

1. Cambiar a provider más rápido (ej. Groq para casos simples)
2. Implementar caching de respuestas
3. Auditar skills lentos y optimizar
4. Escalar VPS verticalmente

9.3 Problemas de Seguridad

Síntoma: Skill sospechoso instalado

Acciones inmediatas:

1. Deshabilitar skill: remover de configuración
2. Revisar logs para actividad anómala
3. Rotar todas las API keys y secrets
4. Auditar otros skills instalados
5. Reportar a ClawHub si es skill del marketplace

---

10. Recursos Adicionales

10.1 Documentación Oficial

- OpenClaw Docs: https://openclaw.ai/docs
- ClawHub: https://clawhub.biz
- GitHub Repository: https://github.com/openclaw/openclaw

10.2 Comunidad y Soporte

- Discord oficial de OpenClaw
- Reddit: r/OpenClawCentral
- Stack Overflow tag: `openclaw`

10.3 Herramientas Complementarias

- **SwarmClaw**: Orquestación multi-agente
- **LangGraph**: Workflows complejos con checkpointing
- **CrewAI**: Equipos de agentes especializados
- **Qdrant/Weaviate**: Vector stores para memoria semántica

Referencias

[1] OpenClaw Documentation. (2026). Install OpenClaw. https://openclaw-ai.com/en/docs/install/index/

[2] OpenClaw Docs. (2026). Install - OpenClaw Docs. https://docs.openclaw.ai/install

[3] Tencent Cloud. (2026, febrero 27). 2026 Beginner's Guide to Deploying OpenClaw and Best Practices. https://www.tencentcloud.com/techpedia/139433

[4] Delx. (2026, febrero 18). OpenClaw Best Practices (2026): 7 Rules for Production Agents. https://delx.ai/openclaw/openclaw-best-practices

[5] Tencent Cloud. (2026, marzo 2). OpenClaw Automation Best Practices: Process Design. https://www.tencentcloud.com/techpedia/140636

[6] Lao Zhang AI. (2026, febrero 7). OpenClaw Installation and Deployment: The Complete Guide. https://blog.laozhang.ai/en/posts/openclaw-installation-deployment-guide

[7] OpenClaw AI. (2026). OpenClaw AI Online Deployment Guide. https://openclawai.net/deployment

[8] ClawHub Biz. (2026). ClawHub: OpenClaw Skills Resource Hub. https://clawhub.biz

[9] TechCrunch. (2026). Top OpenClaw Skills of 2026. https://techcrunch.com/openclaw-skills-2026

[10] GitHub. (2026). SKILL.md Specification. https://github.com/openclaw/skill-specification

[11] OpenClaw Community. (2026). Building Custom Skills Guide. https://community.openclaw.ai/skills

[12] Webclaw Documentation. (2026). Automatic UI Generation. https://webclaw.dev/docs

[13] OpenClaw Blog. (2026). Introducing Webclaw: UI from SKILL.md. https://blog.openclaw.ai/webclaw-launch

[14] SwarmClaw Docs. (2026). Multi-Agent Orchestration. https://swarmclaw.ai/docs

[15] Redis. (2026, febrero 2). Top AI Agent Orchestration Platforms in 2026. https://redis.io/blog/ai-agent-orchestration-platforms/

[16] IBL.ai. (2026, febrero 24). Securing Autonomous Agents: OpenClaw, IronClaw and NanoClaw. https://ibl.ai/blog/securing-autonomous-agents

[17] Security Week. (2026). NanoClaw and IronClaw: Security-First AI Agents. https://securityweek.com/nanoclaw-ironclaw-2026

[18] LinkedIn. (2026, febrero 17). Deploy OpenClaw Securely with HTTPS on Hostinger VPS. https://www.linkedin.com/posts/mikemurphyco_aihandyman-activity-7429857438184493056-Bw6P

[19] Tailscale. (2026). Tailscale Serve for Self-Hosted Apps. https://tailscale.com/kb/1312/serve

[20] Team400 AI. (2026). Running OpenClaw in Production - Lessons Learned. https://team400.ai/blog/2026-02-openclaw-production-enterprise

[21] Valletta Software. (2026, febrero 26). OpenClaw Architecture & Setup Guide (2026). https://vallettasoftware.com/blog/post/openclaw-2026-guide

[22] Towards AI. (2026, marzo 2). OpenClaw Complete Tutorial 2026: Setup, Skills, Memory. https://pub.towardsai.net/openclaw-complete-guide-setup-tutorial-2026-14dd1ae6d1c2

[23] Valletta Software. (2026, marzo 3). How to Install OpenClaw: Mac Mini, Docker, VPS, Gateway. https://vallettasoftware.com/blog/post/how-to-install-openclaw-mac-mini-docker-vps-gateway-2026

[24] Cherry Servers. (2026, marzo 3). How to Install OpenClaw on Linux VPS. https://www.cherryservers.com/blog/install-openclaw-on-linux-vps

[25] YouTube. (2026, febrero 9). How To Install OpenClaw on Hostinger VPS (Docker). https://www.youtube.com/watch?v=XvEDmYObHaI

[26] Hostinger. (2026, enero 28). Cómo configurar OpenClaw en VPS con Docker y Telegram. https://www.hostinger.com/es/tutoriales/como-configurar-openclaw-en-un-vps

[27] DigitalOcean. (2026, enero 28). How to Run OpenClaw with DigitalOcean. https://www.digitalocean.com/community/tutorials/how-to-run-openclaw

[28] Global Builders Club. (2026, febrero 1). How to Actually Run OpenClaw: Security and Best Practices. https://www.globalbuilders.club/blog/openclaw-best-practices

[29] DreamFactory. (2026, febrero 23). Running OpenClaw Responsibly in Production. https://www.linkedin.com/pulse/running-openclaw-responsibly-production-dreamfactory-0frjc

[30] Hetzner Community. (2026). Setup Dokploy on your VPS - Tutorial. https://community.hetzner.com/tutorials/setup-dokploy-on-your-vps/

[31] DevMystify. (2025, septiembre 16). How to set up Dokploy on Digital Ocean. https://devmystify.com/tutorials/how-to-set-up-dokploy-on-digital-ocean-and-deploy-an-application-with-docker-compose

[32] Dokploy. (2026). Dokploy - Deploy your applications with ease. https://dokploy.com

[33] CNBC. (2026, febrero 2). OpenClaw: The AI agent generating buzz. https://www.cnbc.com/2026/02/02/openclaw-open-source-ai-agent-rise

[34] TechCrunch. (2026). OpenClaw Security Considerations. https://techcrunch.com/openclaw-security-2026

[35] Penligent AI. (2026, marzo 3). ClawHub Skill Scanning and Supply-Chain Security. https://www.penligent.ai/hackinglabs/openclaw-virustotal

[36] Security Research. (2026). 300+ Malicious Skills Found in ClawHub Marketplace. https://securityresearch.com/clawhub-malware-2026

[37] OpenClaw Community. (2026). Top Rated Skills 2026. https://community.openclaw.ai/top-skills

[38] OpenClaw Lab. (2026, febrero 25). Install - OpenClaw Open Source AI Assistant. https://openclawlab.com/en/docs/install/